

PROGRAMAÇÃO ORIENTADA A OBJETOS



Construtores e herança

Victor de Andrade Machado

OBJETIVOS DE APRENDIZAGEM

- > Definir método construtor de uma classe com exemplos de implementação em Java.
- > Explicar os fundamentos de herança em orientação a objetos.
- > Construir um aplicativo com estrutura de herança fazendo uso de construtores para criar objetos.

Introdução

Construtores são métodos especiais utilizados para inicializar objetos de uma classe, enquanto a herança permite a criação de uma hierarquia de classes, em que uma subclasse derivada herda atributos e comportamentos de uma classe-base. A utilização adequada de construtores é essencial para garantir a correta inicialização de objetos, estabelecendo seu estado inicial de maneira consistente. Além disso, a herança é uma poderosa técnica da programação orientada a objetos, que promove a reutilização de código e a organização hierárquica de classes, facilitando a modelagem de sistemas complexos.

Neste capítulo, aprenderemos a definir construtores em Java e a utilizá-los para criar objetos, além de entender os fundamentos da herança e como aplicá-la de forma efetiva em nossos projetos.

Definição de método construtor

Os construtores desempenham um papel crucial na inicialização de objetos em Java, permitindo que sejam definidos os valores iniciais dos atributos de uma classe. A herança, por sua vez, possibilita a reutilização de código e a criação de relacionamentos entre classes, o que promove a organização e a modularidade em um sistema orientado a objetos (HORSTMANN, 2009).

Os construtores são métodos especiais que têm o mesmo nome da classe em que estão definidos, e são responsáveis por inicializar os objetos dessa classe. Eles são chamados automaticamente quando um objeto é criado e podem receber parâmetros para definir os valores iniciais dos atributos (ORACLE, 2023).

Alguns padrões básicos dos construtores são:

- recebem o mesmo nome da classe;
- não possuem tipo de retorno;
- podem ser sobrecarregados, permitindo a criação de vários construtores com diferentes assinaturas;
- podem conter instruções para inicializar os atributos do objeto.

A sintaxe básica de um construtor em Java é mostrada na Figura 1.

```
public class MinhaClasse {  
  
    public MinhaClasse() {  
        // Instruções de inicialização  
    }  
  
}
```

Figura 1. Sintaxe de um construtor em Java.

Os construtores são métodos especiais chamados automaticamente quando um objeto é criado a partir de uma classe. Eles têm a responsabilidade de inicializar os atributos e estabelecer o estado inicial do objeto. Os construtores apresentam as seguintes características (DEITEL; DEITEL, 2016):

- **Nome do método** — o nome do construtor é sempre o mesmo nome da classe em que ele está declarado. No exemplo da Figura 1, `MinhaClasse()` é o nome do método construtor em questão.
- **Ausência de tipo de retorno** — ao contrário dos demais métodos, os construtores não possuem um tipo de retorno definido, nem mesmo o tipo `void`.
- **Chamada implícita** — quando um objeto é criado, o construtor correspondente é chamado automaticamente para inicializar o objeto. Isso é feito utilizando a palavra-chave `new` seguida do nome da classe e dos parâmetros, se houver.
- **Sobrecarga de construtores** — assim como os demais métodos, os construtores podem ser sobrecarregados, ou seja, uma classe pode ter vários construtores com diferentes assinaturas.

Vejamos a seguir alguns dos motivos pelos quais os construtores são utilizados (FINEGAN; LIGUORI, 2018).

- **Inicialização dos atributos:** os construtores são responsáveis por inicializar os atributos de um objeto com valores adequados. Assim, permitem que os objetos sejam criados com estados consistentes e prontos para serem utilizados.
- **Encapsulamento:** ao definir construtores, é possível controlar o acesso aos atributos de uma classe. Se os atributos forem privados, o construtor pode ser utilizado para atribuir valores a esses atributos de forma segura, evitando acesso direto externo.
- **Flexibilidade na criação de objetos:** os construtores podem receber parâmetros que permitem a personalização dos objetos durante a criação. Desse modo, é possível criar objetos com diferentes configurações iniciais, tornando o código mais flexível e adaptável a diferentes cenários.
- **Manutenibilidade e legibilidade do código:** o uso de construtores bem definidos torna o código mais legível e facilita a manutenção. Ao criar objetos por meio de construtores, o código se torna mais claro, pois fica evidente quais são os valores iniciais dos atributos.
- **Aplicação de lógica adicional:** os construtores podem conter lógica adicional, além da simples inicialização de atributos. Isso permite a execução de ações específicas durante a criação de um objeto, como a validação de parâmetros ou a realização de cálculos prévios.

Vamos supor que queremos criar uma classe que possua um método construtor, caracterizada por contar com os atributos `marca` e `modelo`. Dessa forma, podemos representar essa classe conforme mostrado na Figura 2.

```
public class Carro {  
  
    private String marca;  
    private String modelo;  
  
    public Carro() {  
        marca = "Sem marca";  
        modelo = "Sem modelo";  
    }  
  
    public Carro(String marca, String modelo) {  
        this.marca = marca;  
        this.modelo = modelo;  
    }  
}
```

Figura 2. Classe em Java utilizando construtor.

Nesse exemplo, temos uma classe `Carro` com dois construtores. O primeiro construtor não recebe parâmetros e inicializa os atributos `marca` e `modelo` com valores-padrão. O segundo construtor recebe os valores da marca e do modelo como parâmetros e atribui-os aos atributos correspondentes usando a palavra-chave `this`.

Para definir um construtor numa classe, basta criar um método com o mesmo nome da classe. Assim, é possível ter vários construtores na mesma classe, desde que tenham diferentes assinaturas (número ou tipos de parâmetros) (DEITEL; DEITEL, 2016).

Para criar um objeto utilizando um construtor, utiliza-se a palavra-chave `new` seguida do nome da classe e dos parâmetros, se houver. O construtor correspondente será chamado e o objeto será inicializado, conforme mostrado na Figura 3.

```
Carro meuCarro = new Carro();  
Carro outroCarro = new Carro("Ford", "Fiesta");
```

Figura 3. Criação de um objeto utilizando construtor.

Vejamos outro exemplo usando o construtor. Observe o código apresentado na Figura 4.

```
public class Retangulo {  
    private double altura;  
    private double largura;  
  
    public Retangulo() {  
        altura = 1.0;  
        largura = 1.0;  
    }  
  
    public Retangulo(double altura, double largura) {  
        this.altura = altura;  
        this.largura = largura;  
    }  
}
```

Figura 4. Classe e chamada a um construtor.

Nesse exemplo, a classe `Retangulo` possui dois construtores. O primeiro inicializa a altura e largura com valores-padrão de 1.0, enquanto o segundo permite a definição dos valores de altura e largura ao criar o objeto.



Fique atento

Ao trabalhar com construtores em Java, é importante estar ciente de alguns pontos para evitar possíveis armadilhas. Primeiramente, tome cuidado ao lidar com **lógica complexa** dentro dos construtores. É recomendável que os construtores sejam simples e tenham a responsabilidade principal de **inicializar os atributos** do objeto. Evite realizar operações demoradas ou de grande processamento no construtor, pois isso pode prejudicar o desempenho do programa. Além disso, evite vazamentos de memória ao utilizar construtores, garantindo que todos os recursos alocados, como conexões com bancos de

dados ou arquivos abertos, sejam liberados adequadamente. Por fim, tenha cuidado ao lidar com construtores **privados** ou **restritos**, pois o uso inadequado pode dificultar a criação de instâncias da classe ou levar a comportamentos inesperados. Certifique-se de entender claramente o **propósito** e a **utilização** desses construtores antes de implementá-los. Mantendo-se atento a esses pontos, você poderá utilizar construtores de forma efetiva e evitar problemas comuns associados a eles.

Na próxima seção, veremos outro conceito da programação orientada a objetos diretamente ligado ao uso de construtores, em que parte de sua implementação é feita em classes-pai, ou seja, em que o código herdará um conjunto de elementos.

Herança em programação orientada a objetos

A herança é um dos pilares fundamentais da programação orientada a objetos. Ela permite que uma classe derive características de outra classe, aproveitando atributos e métodos já definidos. A classe derivada, também chamada de **subclasse** ou **classe-filha**, herda as propriedades da classe-base, também conhecida como **superclasse** ou **classe-pai**. Dentre as vantagens da herança em programação orientada a objetos, podemos destacar:

- Reutilização de código — a herança permite aproveitar a implementação existente numa classe-pai, evitando duplicação de código.
- Organização e modularidade — fazendo uso de herança, é possível criar hierarquias de classes que refletem a estrutura do domínio do problema, tornando o código mais organizado e modular.
- Polimorfismo — a herança possibilita o uso de polimorfismo, permitindo que objetos de diferentes classes derivadas sejam tratados de maneira uniforme.

Na herança, temos a relação entre uma superclasse e suas subclasses. A superclasse é a classe-base da hierarquia e contém características comuns a todas as subclasses. As subclasses, por sua vez, herdam essas características e podem adicionar novos atributos e comportamentos específicos (DEITEL; DEITEL, 2016).

Suponha que estamos desenvolvendo um sistema de gerenciamento de funcionários de uma empresa. Podemos ter uma classe-base chamada `Funcionario`, que contém atributos e métodos comuns a todos os funcionários. Em seguida, podemos ter subclasses como `Gerente`, `Programador` e `Analista`, que recebem herança da classe `Funcionario` e adicionam atributos e comportamentos específicos de cada cargo.

Herança em Java

Para declarar uma classe como subclasse de outra classe em Java, utiliza-se a palavra-chave `extends`. A subclasse herda os atributos e métodos da superclasse, conforme exemplificado na Figura 5.

```
public class Subclasse extends Superclasse {  
    // Atributos e métodos da subclasse  
}
```

Figura 5. Estrutura de herança em Java.

Ao herdar uma classe, os membros da superclasse podem ter diferentes modificadores de acesso. Os membros **públicos** da superclasse podem ser acessados diretamente pela subclasse. Os membros **protegidos** também podem ser acessados, mesmo que estejam em pacotes diferentes. Já os membros **privados** da superclasse não são acessíveis diretamente pela subclasse.



Saiba mais

A herança e o uso de construtores são conceitos fundamentais na programação orientada a objetos em Java. Se você deseja aprofundar seus conhecimentos sobre esses tópicos, existem muitos recursos disponíveis para ajudá-lo. Além da documentação oficial da Oracle sobre Java, você pode explorar tutoriais, livros e cursos *on-line* que abordam a programação orientada a objetos em Java, com ênfase em herança e construtores. Isso permitirá que você compreenda melhor os conceitos, a sintaxe e as melhores práticas associadas a esses tópicos.

Uma subclasse pode **sobrescrever** um método da superclasse, fornecendo uma implementação diferente (SCHILDT, 2015). Para isso, utiliza-se a anotação `@Override` antes da declaração do método na subclasse. A **sobrescrita de método** permite modificar o comportamento do método herdado (FINEGAN; LIGUORI, 2018). Vejamos um exemplo de herança, mostrado na Figura 6, com a utilização da sobrescrita de método.

```
public class Animal {  
    protected String nome;  
  
    public void emitirSom() {  
        System.out.println("Animal emitindo som...");  
    }  
}  
  
public class Cachorro extends Animal {  
    public void latir() {  
        System.out.println("Au Au!");  
    }  
  
    @Override  
    public void emitirSom() {  
        System.out.println("O cachorro está latindo.");  
    }  
}
```

Figura 6. Exemplo de herança com sobrecarga de método.

No exemplo da Figura 6, temos uma classe `Animal` como superclasse e uma classe `Cachorro` como subclasse. A classe `Cachorro` herda o atributo `nome` e o método `emitirSom` da classe `Animal` e adiciona o método `latir`. O método `emitirSom` é sobrescrito na classe `Cachorro` para fornecer uma implementação específica para os cachorros. O diagrama de classes de nosso exemplo é mostrado na Figura 7.

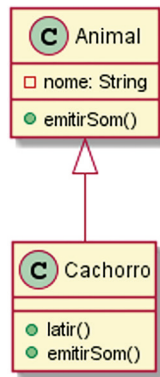


Figura 7. Diagrama de classes da classe `Animal`.

No diagrama, a seta aponta da classe `Cachorro` para a classe `Animal`, indicando que `Cachorro` é uma subclasse de `Animal` e herda seus atributos e métodos. A notação “+” indica membros públicos e a notação “-” indica membros privados.

Agora que já examinamos exemplos de herança e construtor, na próxima seção veremos um exemplo mais completo envolvendo esses dois conceitos de orientação a objetos.

Exemplo prático em Java envolvendo construtor e herança

Vamos construir um aplicativo que simula um sistema de vendas numa loja. Teremos uma classe-base chamada `Produto`, que conterá informações básicas sobre cada produto, como nome, preço e descrição. Em seguida, teremos subclasses específicas para diferentes tipos de produtos, como `Livro`, `Eletrônico` e `Roupa`. Primeiramente, apresentamos na Figura 8 o diagrama de classe referente a esse exemplo prático.

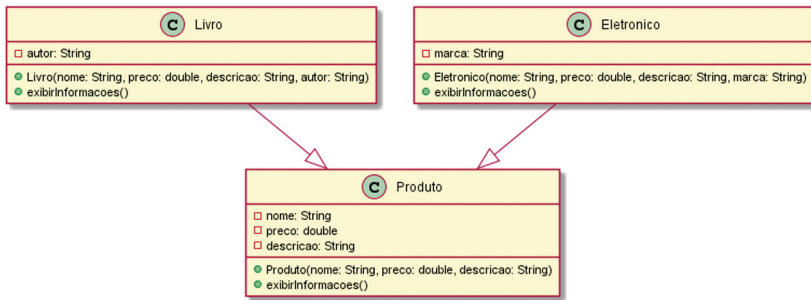


Figura 8. Diagrama para classe `Produto`.

O diagrama mostra a classe `Produto` com três atributos: `nome`, `preco` e `descricao`. No diagrama, a seta aponta das classes `Livro` e `Eletronico` para a classe `Produto`, indicando que `Livro` e `Eletronico` são subclasses de `Produto` e herdam seus atributos e métodos. A Figura 9 mostra a implementação das classes em questão.

```

public class Produto {

    protected String nome;
    protected double preco;
    protected String descricao;

    public Produto(String nome, double preco, String descricao) {
        this.nome = nome;
        this.preco = preco;
        this.descricao = descricao;
    }

    public void exibirInformacoes() {
        System.out.println("Nome: " + nome);
        System.out.println("Preço: " + preco);
        System.out.println("Descrição: " + descricao);
    }
}

```

Figura 9. Classe `Produto`.

No exemplo, temos a classe `Produto` como classe-base, que possui um construtor para inicializar os atributos `nome`, `preco` e `descricao`. A classe `Produto` também possui um método `exibirInformacoes` para mostrar os detalhes do produto. Observe agora, na Figura 10, as subclasses `Livro` e `Eletronico`.

```
public class Livro extends Produto {  
    private String autor;  
  
    public Livro(String nome, double preco, String descricao, String  
autor) {  
        super(nome, preco, descricao);  
        this.autor = autor;  
    }  
  
    @Override  
    public void exibirInformacoes() {  
        super.exibirInformacoes();  
        System.out.println("Autor: " + autor);  
    }  
}  
  
//Nova subclasse (em outro arquivo)  
public class Eletronico extends Produto {  
    private String marca;  
  
    public Eletronico(String nome, double preco, String descricao,  
String marca) {  
        super(nome, preco, descricao);  
        this.marca = marca;  
    }  
  
    @Override  
    public void exibirInformacoes() {  
        super.exibirInformacoes();  
        System.out.println("Marca: " + marca);  
    }  
}
```

Figura 10. Subclasses `Livro` e `Eletronico`.

As subclasses `Livro` e `Eletronico` recebem herança da classe `Produto` e adicionam atributos específicos (`autor` e `marca`, respectivamente) a partir de seus construtores. Essas subclasses também sobrescrevem o método `exibirInformacoes` para mostrar informações adicionais.

Agora vamos criar objetos utilizando os construtores definidos nas classes, dentro do método `main` da classe principal, que chamaremos de `Main`, conforme ilustra a Figura 11.

```
public class Main {  
    public static void main(String[] args) {  
        Livro = new Livro("Dom Quixote", 39.90, "Clássico da  
literatura", "Miguel de Cervantes");  
        Eletronico = new Eletronico("Smartphone", 1999.90, "Telefone  
inteligente", "Samsung");  
  
        livro.exibirInformacoes();  
        eletronico.exibirInformacoes();  
    }  
}
```

Figura 11. Classe `Main`.

Nesse exemplo, criamos um objeto `Livro` chamado `livro` e um objeto `Eletronico` chamado `eletronico`. Utilizamos os construtores específicos de cada classe para inicializar os objetos com valores adequados. Ao executar o código anterior, teremos a saída mostrada na Figura 12.

```
Nome: Dom Quixote  
Preço: 39.9  
Descrição: Clássico da literatura  
Autor: Miguel de Cervantes  
  
Nome: Smartphone  
Preço: 1999.9  
Descrição: Telefone inteligente  
Marca: Samsung
```

Figura 12. Saída da execução do sistema.

Neste capítulo, exploramos os conceitos de construtores e herança em Java. Aprendemos a definir construtores numa classe e a utilizá-los para criar objetos, além de entender os fundamentos da herança em orientação a objetos. Também construímos um aplicativo com estrutura de herança e construtores, em que utilizamos classes-base e subclasses para representar produtos num sistema de vendas.

Os construtores são essenciais para a inicialização correta de objetos em Java, permitindo definir os valores iniciais dos atributos de uma classe. A herança, por sua vez, é fundamental para a reutilização de código, a organização e a modularidade num sistema orientado a objetos.

Referências

DEITEL, P.; DEITEL, H. *Java: como programar*. 10. ed. São Paulo: Pearson, 2016.

FINEGAN, E.; LIGUORI, R. *OCA Java SE 8 Programmer I: guia de estudos (Exame 1Z0-808)*. 3. ed. Porto Alegre: Bookman, 2018.

HORSTMANN, C. *Conceitos de computação com Java*. 5. ed. Porto Alegre: Bookman, 2009.

ORACLE. Java platform, standard edition 8: API specification. *Oracle*, 2023. Disponível em: <https://docs.oracle.com/javase/8/docs/api/>. Acesso em: 7 jun. 2023.

SCHILDT, H. *Java para iniciantes: crie, compile e execute programas Java rapidamente*. 6. ed. Porto Alegre: Bookman, 2015.

Leituras recomendadas

GOODRICH, M. T.; TAMASSIA, R. *Estruturas de dados e algoritmos em Java*. Porto Alegre: Bookman, 2013.

SCHACH, S. R. *Engenharia de software: os paradigmas clássico e orientado a objetos*. 7. ed. Porto Alegre: AMGH, 2009.



Fique atento

Os links para sites da web fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integridade das informações referidas em tais links.

Conteúdo:

sagah⁺